

**Professores:**

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

# WEB STORAGE



# O Problema: Dados que Somem

Nas aulas anteriores, o TechFood ganhou interatividade. Mas experimente clicar em "Pedir Agora" e depois recarregar a página.

## O pedido sumiu. Por quê?

Tudo que criamos com JavaScript — variáveis, arrays, objetos — vive apenas na **memória RAM** do navegador. Quando a página fecha ou recarrega, essa memória é apagada completamente.

**Analogia:** É como escrever numa lousa e apagar tudo ao fechar a sala. O Web Storage é o caderno — o que você escreve lá **fica**, mesmo depois de fechar.

# O que é Web Storage?

Web Storage é uma API nativa do navegador que permite **guardar dados localmente**, no próprio computador do usuário, sem precisar de servidor ou banco de dados.

São dois tipos:

|                                  | localStorage                    | sessionStorage                          |
|----------------------------------|---------------------------------|-----------------------------------------|
| <b>Dura até</b>                  | O usuário limpar manualmente    | Fechar a aba/navegador                  |
| <b>Compartilhado entre abas?</b> | ✓ Sim                           | ✗ Não                                   |
| <b>Uso típico</b>                | Preferências, carrinho, pedidos | Formulários em etapas, login temporário |

**Lembre-se:** Ambos guardam dados **só no navegador do usuário** — nenhum outro dispositivo acessa. Para dados que precisam viajar entre dispositivos, usamos um servidor (o que veremos nas aulas de Fetch/API).

# Os 4 Métodos do localStorage

// 1. SALVAR um dado

```
localStorage.setItem("chave", "valor");
```

// 2. LER um dado

```
const dado = localStorage.getItem("chave");
```

// 3. REMOVER um dado específico

```
localStorage.removeItem("chave");
```

// 4. LIMPAR tudo

```
localStorage.clear();
```

# Os 4 Métodos do localStorage

## Exemplo

**Importante:** O localStorage só aceita **strings**. Números, booleanos e objetos precisam ser convertidos antes de salvar.

// Número → vira string ao salvar, converter ao ler

```
localStorage.setItem("preco", 45.00);
```

```
const preco = Number(localStorage.getItem("preco"));
```

// Booleano → comparar como string ao ler

```
localStorage.setItem("logado", true);
```

```
const logado = localStorage.getItem("logado") === "true";
```



# Salvando Objetos: JSON.stringify e JSON.parse

O localStorage só aceita strings. Para salvar objetos e arrays usamos:

```
// Salvar objeto → converte para string
```

```
JSON.stringify(objeto)
```

```
// Ler string → converte de volta para objeto
```

```
JSON.parse(string)
```

**Analogia:** **JSON.stringify** é tirar uma foto do objeto. **JSON.parse** é revelar a foto de volta

# Limitações do Web Storage

Antes de usar o Web Storage em produção, é importante conhecer suas limitações:

| Limitação            | Detalhe                                                              |
|----------------------|----------------------------------------------------------------------|
| <b>Capacidade</b>    | ~5MB por domínio — suficiente para dados textuais, não para imagens  |
| <b>Só strings</b>    | Objetos precisam de <b>JSON.stringify</b> e <b>JSON.parse</b>        |
| <b>Mesmo domínio</b> | <b>localhost</b> não acessa dados de <b>techfood.com</b>             |
| <b>Sem segurança</b> | Qualquer JS da página lê o storage — nunca salve senhas ou tokens    |
| <b>Sem expiração</b> | Os dados ficam até o usuário limpar — não há expiração automática    |
| <b>Síncrono</b>      | Bloqueia a thread principal — evite salvar objetos gigantes em loops |

# Padrão de Uso: Lista no localStorage

Padrão **ler** → **modificar** → **salvar** — base de qualquer operação com listas:

// ADICIONAR (Create)

```
const lista = JSON.parse(localStorage.getItem("chave") || "[]");
```

// O || "[]" evita erro quando a chave ainda não existe.

```
lista.push(novoItem);
```

```
localStorage.setItem("chave", JSON.stringify(lista));
```

// REMOVER por índice (Delete)

```
lista.splice(indice, 1);
```

```
localStorage.setItem("chave", JSON.stringify(lista));
```

**Atenção: splice(indice, 1)** remove da **memória** — diferente do **remove()** da Aula 7 que apagava só da tela. Se recarregar a página, o item não volta.



# O Projeto Cresceu: global.js

O TechFood agora terá três páginas (cardápio, pedidos, cadastro), disso surgiu um problema:

A **saudação** e a **data do rodapé** estavam no **aula7.js** — que só carregava no **index.html**. As outras páginas ficariam sem.

**Solução:** separar o que é de **todas as páginas** num arquivo próprio.

```
src/scripts/  
├─ global.js    ← saudação, footer, menu mobile (todas as páginas)  
├─ main.js      ← lógica do cardápio (index.html)  
├─ pedidos.js   ← lógica dos pedidos (pedidos.html)  
└─ cadastro.js ← lógica do cadastro (cadastro.html) — próxima aula
```

**Lembre-se:** O **global.js** deve ser carregado **antes** dos outros scripts em cada página — ele define funções que os outros arquivos podem precisar.

# Código Solto vs DOMContentLoaded

Na Aula 7, o código estava **solto** — rodava direto ao carregar o arquivo, sem garantia de que o HTML existia.

**Aula 8:** tudo que acessa o DOM vira função e é chamado dentro do **DOMContentLoaded**:

```
document.addEventListener("DOMContentLoaded", function () {  
  inicializarHoverCards();  
  inicializarSubtotal();  
  inicializarVitrine();  
});
```

Cada função cuida de uma responsabilidade — e só roda quando o HTML está pronto.

**Lembre-se: DOMContentLoaded** garante que todos os elementos HTML existam antes do JS tentar acessá-los.

- ✓ **Web Storage** = API do navegador para guardar dados localmente, sem servidor.
- ✓ **localStorage** = Persiste até o usuário limpar. Compartilhado entre abas.
- ✓ **sessionStorage** = Apaga ao fechar a aba. Não compartilhado.
- ✓ **setItem / getItem** = Salvar e ler dados. Só aceita strings.
- ✓ **removeItem / clear** = Remover uma chave ou limpar tudo. Prefira **removeItem** — mais preciso.
- ✓ **JSON.stringify** = Converte objeto → string para salvar no storage.
- ✓ **JSON.parse** = Converte string → objeto ao ler do storage.
- ✓ **Padrão seguro** = **JSON.parse(localStorage.getItem("chave") || "[]")** evita erros.
- ✓ **Padrão ler → modificar → salvar** = Base de qualquer operação com listas no storage.
- ✓ **splice(indice, 1)** = Remove da memória (array + storage). Diferente do **remove()** que apaga só da tela.
- ✓ **global.js** = Arquivo compartilhado por todas as páginas — saudação, footer, menu.
- ✓ **DOMContentLoaded** = Garante que o HTML existe antes do JS tentar acessá-lo.

# Portal EcoCycle

1

## Primeiros passos no localStorage

Contexto: Portal EcoCycle (Notícias de Ecologia)

No console do navegador (F12), pratique os 4 métodos:

1. Salve seu nome: **localStorage.setItem("meuNome", "SeuNome")**
2. Leia de volta: **localStorage.getItem("meuNome")**
3. Abra outra aba do mesmo site e leia novamente — o dado ainda está lá?
4. Remova: **localStorage.removeItem("meuNome")**
5. Tente ler novamente — o que retorna?

**Bonus:** Salve um número e leia de volta. Depois salve **true** (booleano) e leia. O que você recebe? Como converter de volta para o tipo correto?

# Portal EcoCycle

2

## Salvando e lendo objetos

Contexto: Portal EcoCycle (Notícias de Ecologia)

No arquivo **eco.js** do projeto EcoCycle:

1. Crie um objeto **preferencias** com: **{ tema: "claro", ultimoValorPapel: 0 }**
2. Salve no localStorage com a chave **"ecocycle\_prefs"**
3. Recarregue a página e leia o objeto de volta
4. No evento **input** do simulador, atualize **ultimoValorPapel** e salve novamente
5. Recarregue e verifique: o último valor digitado continua lá?



# Portal EcoCycle

3

## Lista de notícias sugeridas

Contexto: Portal EcoCycle (Notícias de Ecologia)

Ainda no EcoCycle — persistindo as notícias sugeridas pelo usuário (Exercício da Aula 7):

1. Ao adicionar uma notícia com **adicionarNoticia()**, salve também no localStorage com a chave "**ecocycle\_noticias**" usando o padrão ler → push → salvar
2. Ao carregar a página (**DOMContentLoaded**), leia as notícias salvas e chame **adicionarNoticia()** para cada uma
3. Teste: adicione 2 notícias, recarregue — elas continuam aparecendo?



# Portal EcoCycle

## 4

### Removendo dados

Contexto: Portal EcoCycle (Notícias de Ecologia)

No EcoCycle, adicione um botão "Limpar Dados Salvos" na página:

1. Crie o botão no HTML com **id="btn-limpar-storage"**
2. No JS, adicione um **addEventListener** de **click**
3. Use **localStorage.removeItem("ecocycle\_noticias")** e **localStorage.removeItem("ecocycle\_prefs")** para remover só as chaves do EcoCycle
4. Após remover, limpe a lista de notícias na tela
5. **Desafio:** Qual a diferença de **removeItem("chave")** vs **clear()**? Quando usar cada um?

# Portal EcoCycle

5

## EcoCycle com memória completa

Contexto: Portal EcoCycle (Notícias de Ecologia)

Integrando tudo — o EcoCycle deve ter memória completa:

1. **Preferências persistidas:** o último valor do simulador é restaurado ao carregar a página
2. **Notícias persistidas:** notícias sugeridas sobrevivem ao recarregamento
3. **Contador de visitas:** a cada abertura da página, incremente um contador no localStorage e exiba no header: "Bem-vindo de volta! Esta é sua Nª visita."
4. **Botão limpar tudo:** remove todas as chaves do EcoCycle e restaura o estado inicial
5. **Verificação:** abra o DevTools (F12 → Application → Local Storage) e confirme que todas as chaves estão sendo criadas e removidas corretamente



MOZILLA. **Window.localStorage**. MDN Web Docs. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/API/Window/localStorage>. Acesso em: abr. 2026.

MOZILLA. **Web Storage API**. MDN Web Docs. Disponível em: [https://developer.mozilla.org/pt-BR/docs/Web/API/Web\\_Storage\\_API](https://developer.mozilla.org/pt-BR/docs/Web/API/Web_Storage_API). Acesso em: abr. 2026.

MOZILLA. **JSON.stringify()**. MDN Web Docs. Disponível em: [https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global\\_Objects/JSON/stringify](https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify). Acesso em: abr. 2026.

MOZILLA. **JSON.parse()**. MDN Web Docs. Disponível em: [https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global\\_Objects/JSON/parse](https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse). Acesso em: abr. 2026.

ANDRADE, Sidnei da Silva. **Criação e interfaces para web HTML5 e CSS3**. São Paulo: SENAI SP Editora, 2017.

SENAI. **Desenvolvimento Web**. Brasília: SENAI/DN, 2012.



## ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

### Telefone

(11) 2396-1999

### Instagram

@senai.itu

### Facebook

/senaisp.itu

### Site

<https://sp.senai.br/unidade/itu/>